Interrupcions vs Polling

1. Introducció a les Interrupcions vs Polling

En aquest primer apartat, aclarirem què són les interrupcions i el polling, destacant com cada tècnica afecta el rendiment en aplicacions que necessiten una resposta immediata.

Què és el Polling?
El polling és una tècnica on el programa principal consulta constantment l'estat d'un dispositiu o sensor en intervals regulars. Això implica que el sistema verifica contínuament si ha succeït algun esdeveniment. Tot i que és fàcil d'implementar, el polling té desavantatges:
- Consumeix molts recursos perquè manté la CPU ocupada amb comprovacions constants.
- Limita el rendiment, ja que la CPU es dedica a una tasca específica i no pot processar altres instruccions fins que es completa l'esdeveniment o es detecta l'estat desitjat.

Exemple de Polling (sense interrupcions):

```
import RPi.GPIO as GPIO
import time

# Configuració del pin
GPIO.setmode(GPIO.BCM)
SENSOR_PIN = 17
GPIO.setup(SENSOR_PIN, GPIO.IN)

try:
  while True:
    if GPIO.input(SENSOR_PIN):
      print("Esdeveniment detectat!")
    time.sleep(0.1)
finally:
  GPIO.cleanup()
```

Què són les Interrupcions?
Les interrupcions permeten que un programa es 'desvii' temporalment del seu cicle habitual quan es produeix un esdeveniment específic. Això fa que el sistema no hagi d'estar vigilant constantment. Quan el sistema detecta una interrupció, executa el codi associat a l'esdeveniment (funció callback) i després retorna a la seva tasca principal.

Exemple simple d'Interrupció:

```
import RPi.GPIO as GPIO
```

```
import time

# Configuració del pin
GPIO.setmode(GPIO.BCM)
SENSOR_PIN = 17
GPIO.setup(SENSOR_PIN, GPIO.IN)

def esdeveniment_callback(channel):
  print("Interrupció: Esdeveniment detectat!")

GPIO.add_event_detect(SENSOR_PIN, GPIO.RISING, callback=esdeveniment_callback)

try:
  while True:
    print("Esperant...")
    time.sleep(1)
finally:
  GPIO.cleanup()
```

2. Configuració de les Interrupcions amb RPi.GPIO

En aquest apartat, explicarem com configurar les interrupcions en Raspberry Pi utilitzant el mòdul RPi.GPIO, centrant-nos en les opcions específiques que ofereix per detectar esdeveniments i gestionar-los de manera eficient.

Opcions per Detectar Esdeveniments:

1. Tipus d'Esdeveniment:
- **GPIO.RISING:** Interrupció quan hi ha una transició de baix a alt.
- **GPIO.FALLING:** Interrupció quan hi ha una transició d'alt a baix.
- **GPIO.BOTH:** Detecta qualsevol canvi.

2. Debouncing (Anti-rebot):
Permet ignorar senyals rebotejats amb 'bouncetime'.

Exemple de configuració amb debouncing:

```
GPIO.add_event_detect(SENSOR_PIN, GPIO.RISING, callback=esdeveniment_callback, bouncetime=200)
```

Configuració de la Interrupció:

```
import RPi.GPIO as GPIO
import time

# Configuració del pin
GPIO.setmode(GPIO.BCM)
SENSOR_PIN = 17
GPIO.setup(SENSOR_PIN, GPIO.IN)

def encendre_led(channel):
  GPIO.output(LED_PIN, GPIO.HIGH)
  print("Moviment detectat! LED encès.")
  time.sleep(1)
  GPIO.output(LED_PIN, GPIO.LOW)

GPIO.add_event_detect(SENSOR_PIN, GPIO.RISING, callback=encendre_led,
bouncetime=300)
```

3. Exemples Pràctics

En aquest apartat, veurem exemples d'aplicació pràctica d'interrupcions amb RPi.GPIO.

Exemple 1: Interrupció amb un Sensor de Moviment PIR:

```
import RPi.GPIO as GPIO
import time

# Configuració dels pins
GPIO.setmode(GPIO.BCM)
SENSOR_PIN = 17
LED_PIN = 18

# Configuració dels pins com a entrada i sortida
GPIO.setup(SENSOR_PIN, GPIO.IN)
GPIO.setup(LED_PIN, GPIO.OUT)

def encendre_led(channel):
  GPIO.output(LED_PIN, GPIO.HIGH)
  time.sleep(1)
  GPIO.output(LED_PIN, GPIO.LOW)

GPIO.add_event_detect(SENSOR_PIN, GPIO.RISING, callback=encendre_led,
bouncetime=300)
```

Exemple 2: Comptador amb Sensor de Llum:

```
import RPi.GPIO as GPIO
import time

# Configuració del pin
GPIO.setmode(GPIO.BCM)
SENSOR_LUM_PIN = 22
GPIO.setup(SENSOR_LUM_PIN, GPIO.IN)

comptador = 0

def incrementar_comptador(channel):
    global comptador
    comptador += 1
    print(f"Canvi de llum detectat! Comptador: {comptador}")

GPIO.add_event_detect(SENSOR_LUM_PIN, GPIO.BOTH, callback=incrementar_comptador,
bouncetime=200)
```

4. Detecció d'Esdeveniments amb Funcions de Callback

Les funcions callback són funcions que s'executen automàticament quan es detecta un esdeveniment al pin configurat.

Exemple amb Funció Callback:

```
import RPi.GPIO as GPIO
import time

GPIO.setmode(GPIO.BCM)
SENSOR_PIN = 23
MOTOR_PIN = 24
GPIO.setup(SENSOR_PIN, GPIO.IN)
GPIO.setup(MOTOR_PIN, GPIO.OUT)

def activar_motor(channel):
    GPIO.output(MOTOR_PIN, GPIO.HIGH)
    time.sleep(2)
    GPIO.output(MOTOR_PIN, GPIO.LOW)
```

```
GPIO.add_event_detect(SENSOR_PIN, GPIO.RISING, callback=activar_motor,
bouncetime=500)
```

5. Conclusió i Resum dels Conceptes Clau

Aquest temari ha explorat el concepte d'interrupcions en comparació amb el polling, fent servir el mòdul RPi.GPIO.

Resum dels Conceptes Clau:

1. Diferència entre Polling i Interrupció
2. Configuració d'Interrupcions amb RPi.GPIO
3. Ús de Funcions Callback

Aplicacions Pràctiques:

Exemples com Sensor PIR i LED: Entendre com les interrupcions milloren el rendiment respecte al polling.

Conclusió Final:
El control d'interrupcions amb RPi.GPIO proporciona un sistema flexible per desenvolupar aplicacions amb Raspberry Pi.

```python
import RPi.GPIO as GPIO
import time

pinpir = 23
pinled = 24

GPIO.setmode(GPIO.BCM)
GPIO.setup(pinpir, GPIO.IN)
GPIO.setup(pinled, GPIO.OUT)

def intruso_detectado(channel):
    print("Alerta Intruso")
    GPIO.output(pinled, 1)
    time.sleep(0.5)
    GPIO.output(pinled, 0)


GPIO.add_event_detect(pinpir, GPIO.RISING, callback=intruso_detectado)

try:

    while True:
        time.sleep(1)

except KeyboardInterrupt:
    print("Detenido por el usuario")

finally:
    GPIO.cleanup()
'''
apt install python3-rpi-lgpio
Ha calgut aquest comandament per utilitzar aquest codi d'interrupcions
Aquest codi l'hem provat per canviar el paradigma
while True:
    if GPIO.input(pirpin) == 1:
        print("Alerta intrusos")
    time.sleep(1)
'''
```